Gizmonic Cambot: an open, $20 camera that remembers what your robots did

White paper · semantically indexed video for AI monitoring of robots and lab equipment

Gizmonic Cambot · open hardware and open source
Draft 0.1 · October 2026

Summary

Robots fail in ways their logs do not explain. A pipette misses a tip, a gripper drops a part, a mobile base clips a table leg. Video of the event usually exists only if someone happened to be recording, and even then no one wants to scrub through hours of footage to find the four seconds that matter.

Gizmonic Cambot is an open-source camera built from a roughly $12 ESP32-S3 camera board, a microSD card and a 3D-printed case. It records continuously into a rolling buffer on the card, uses a cloud vision model to build a searchable index of what happened, and answers questions in plain language:

> “Show me between 12:00 and 12:30 where the machine picks up a pipette tip.”

The camera replies with the matching moments, a confidence for each, thumbnails and short clips served straight from its SD card. The same functions are available as a REST API and as an MCP server, so a coding agent such as Cursor, or the robot’s own controller, can query the camera directly.

The hardware, case, mounts, firmware and documentation are all open. The case uses a modular mount so the camera can be clamped, stuck magnetically to steel, or attached with any foot a user designs.

The problem

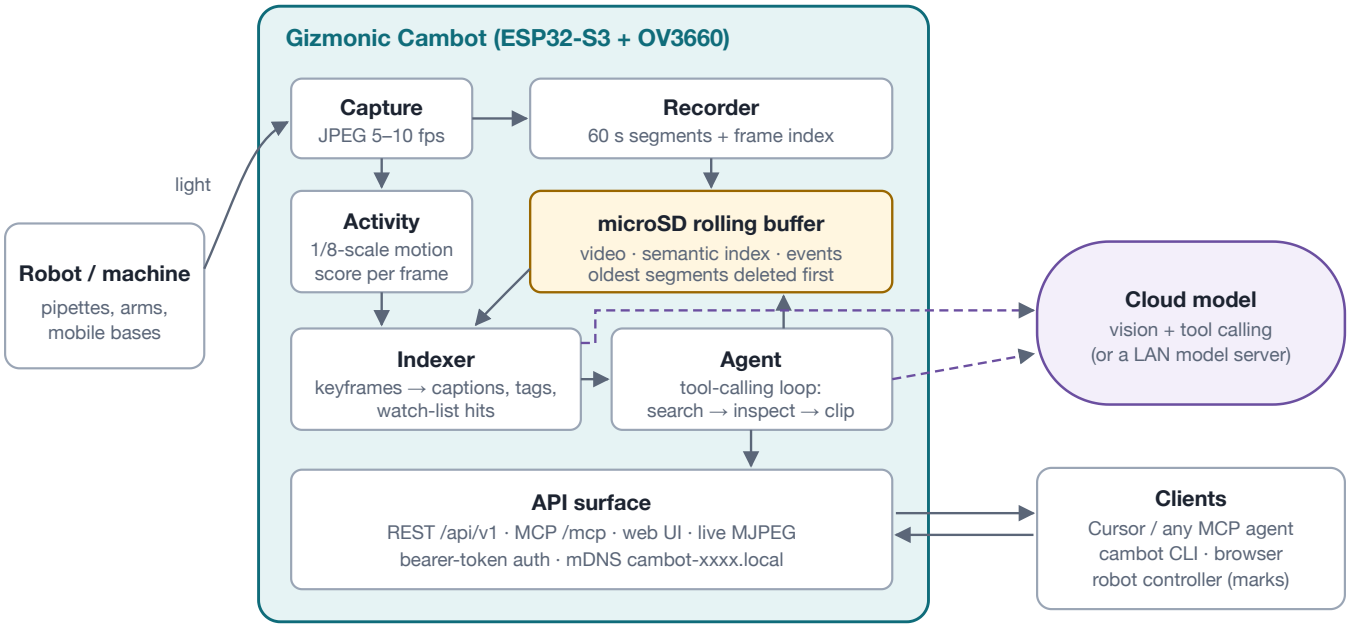
Labs and workshops are filling up with low-cost automation: liquid handlers, desktop arms, 3D printer farms, pick-and-place cells, hobby mobile robots. Three things make visual monitoring of them harder than it should be:

1. **Video is write-only.** Conventional cameras record well but cannot answer “when did X happen?” Finding an event means a human scrubbing a timeline.
2. **Cameras are closed or expensive.** Commercial “AI cameras” lock footage and models into a vendor cloud, cost $50–$300 each, and are designed for doorways rather than robot work cells.
3. **Mounting is an afterthought.** A camera that watches a robot needs to sit on the robot’s frame, a shelf edge, an aluminum extrusion or a steel enclosure, often at an odd angle. Most cameras ship with one tripod thread.

The concept

Cambot is designed to be cheap enough to leave on every machine:

- **Records everything locally.** About 5 frames per second at 800×600 by default, stored as one-minute segments with a per-frame time index. When the card fills, the oldest segments are deleted first. Nothing leaves the camera unless the indexer or a query sends it.
- **Indexes only what matters.** An on-board activity detector scores motion on every frame. Keyframes are sent for captioning only when something is happening, plus one idle keyframe a minute, so an empty room costs almost nothing.
- **Answers questions.** An on-board agent runs a tool-calling loop against a cloud vision model. The model plans the search, the camera does the cheap text search locally, and the model checks only a few candidate windows frame by frame.
- **Speaks to agents.** Every capability is a documented REST endpoint and an MCP tool, so Cursor or any MCP-capable agent can pull clips, ask questions and set up watch lists without custom glue code.



System architecture. Everything left of the cloud runs on the ESP32-S3; the model is only called for captions, verification and planning.

How it works

Capture and the rolling buffer

The OV3660 sensor produces JPEG frames directly, so the ESP32-S3 never encodes video. The recorder appends frames to a one-minute segment file and writes a small index of each frame's timestamp, offset and length. Timestamps come from network time (SNTP) and are stored in UTC; queries are interpreted in the

camera's configured time zone.

Mode	Resolution	Rate	Approx. data rate	32 GB card holds
Default	800×600	5 fps	~0.7 GB/hour	~1.5 days
Detail	1280×720	5 fps	~1.4 GB/hour	~18 hours
Smooth	800×600	10 fps	~1.4 GB/hour	~18 hours
Timelapse	1600×1200	1 fps	~0.4 GB/hour	~3 days

These figures assume typical JPEG sizes at quality 12 and keep 10% of the card free. Busy, detailed scenes produce larger frames.

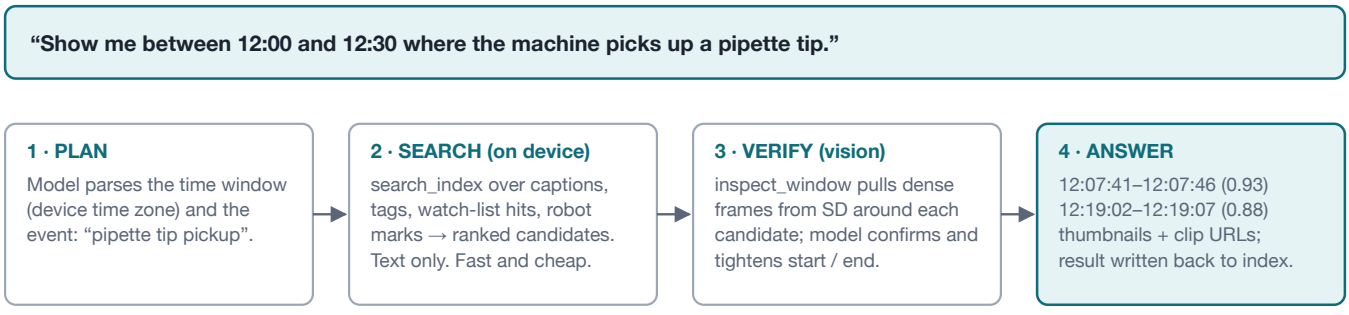
The semantic index

Each keyframe the indexer selects is sent to the vision model with a short **scene profile** written by the user ("This camera watches an Opentrons OT-2 liquid handler. Tip racks are on the left, plates are in the middle.") and an optional **watch list** of named events ("tip pickup", "tip eject", "gripper open", "person in frame"). The model returns structured JSON: a caption, the objects and actions it sees, the robot state, and a yes/no with confidence for each watch-list item.

The camera stores these records on the SD card in hourly files next to the video. Consecutive keyframes that share an action are merged into **events** with a start and end time. An optional phase-2 feature adds small text embeddings for each caption, quantized to a few hundred bytes, so the camera can rank by meaning as well as by keyword.

Robot marks make the index far more precise. A robot controller can call `POST /api/v1/marks` (or pulse an optional trigger input) with a label such as `step 12: pick tip`. Marks are stored on the same timeline as the video, so the camera can answer "show me every step 12" exactly, and compare what the robot claimed to do with what the camera saw.

Answering a question



How a question becomes an answer. Only step 3 sends images to the model.

1. **Plan.** The question goes to the cloud model along with a list of the camera's tools. The model turns "between 12 and 12:30" into a time range and "picks up a pipette tip" into a search.
2. **Search.** The camera searches its own index (captions, tags, watch-list hits, marks and optional embeddings) and returns ranked candidates as text.
3. **Verify.** For the best candidates the camera pulls frames from the SD card, a few seconds either side, and the model confirms the event and narrows its start and end. This also catches events that fell between keyframes.
4. **Answer.** The response lists moments with confidence, thumbnails and clip URLs. Verified results are written back to the index so the next similar question is answered from text alone.

If the index has nothing useful, for example a question about something nobody put on the watch list, the agent can **scan** the window by sampling frames at a fixed interval. Scans are capped by a per-question frame budget, and the agent reports the estimated cost first when a scan would exceed it.

Interfaces

Interface	Used by	What it offers
Web UI at `http://cambot-xxxx.local/`	People	Live view, timeline, ask box, settings
REST `/api/v1/...`	Scripts, robot controllers	Status, frames, clips, index search, ask, marks, config
MCP `/mcp`	Cursor and other agents	The same tools the on-board agent uses
`cambot` CLI	Setup and bulk work	Flash, provision Wi-Fi, discover, ask, pull clips, export MP4

Example uses

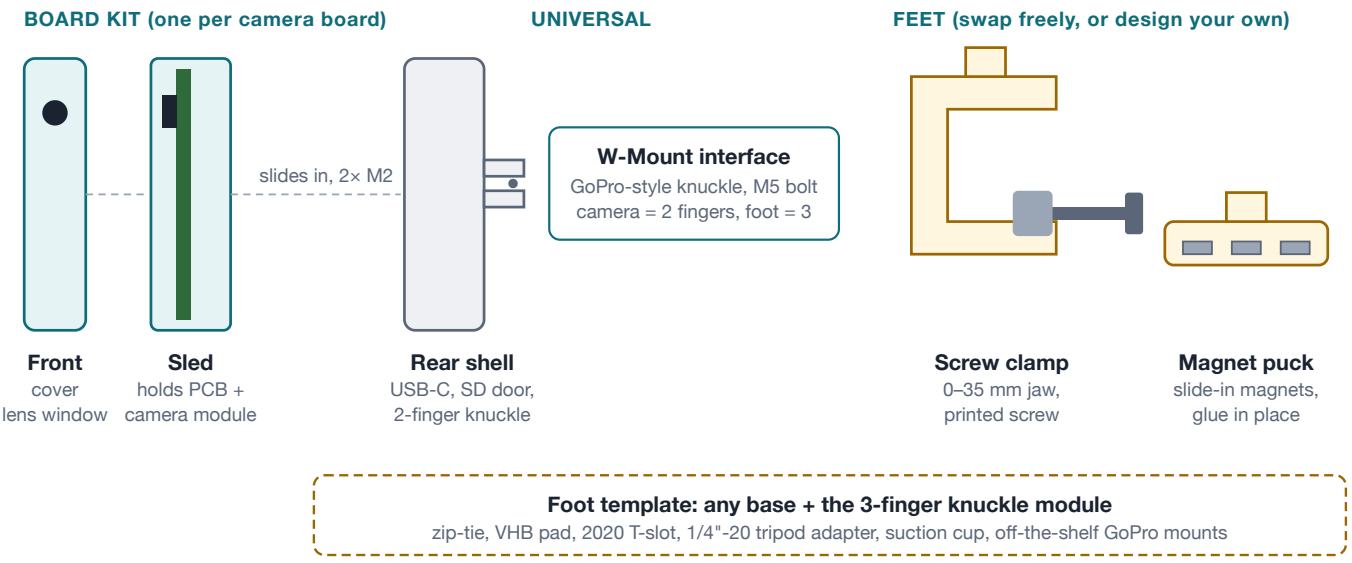
- **Lab automation.** "List every tip pickup this morning that took more than one attempt." "Did anyone open the enclosure during the 2 PM run?"
- **Robot development.** The Lloyd bartender robot that produced this design logs each move as a mark; the camera answers "show me the last five pours" and "where did the car stop short of the tape line?"
- **3D printer farms.** A watch list for "spaghetti", "part detached" and "nozzle blob" turns the index into an alarm feed via the events endpoint.
- **Pick-and-place and QA cells.** Count picks per hour, pull clips of dropped parts, and compare robot marks to what was actually seen.
- **Agent-driven debugging.** A coding agent fixing robot firmware can ask the camera what the robot did after the last flash, instead of asking the human.

Hardware

The reference board is the **FORIOT ESP32-S3-CAM** (Amazon B0GD2BZ9B1), a common layout also sold by Freenove, Goouuu and Keyestudio:

Item	Specification
Module	ESP32-S3-WROOM-1 N16R8: dual-core 240 MHz, 16 MB flash, 8 MB octal PSRAM
Camera	OV3660, 3 MP, on a 24-pin DVP flex cable
Storage	microSD (push-push, rear), 1-bit SDMMC on GPIO 38/39/40
Connectivity	2.4 GHz Wi-Fi 802.11 b/g/n, Bluetooth LE 5
USB	Two USB-C: TTL (CH340 UART, for flashing and logs) and native USB OTG
Indicators	Power LED, WS2812 RGB LED (GPIO 48), user LED (GPIO 2)
Size	62.6 × 28.3 mm PCB (vendor figure)
Power	5 V USB, plan for 0.5 A

Case and modular mounts



The case is split so only two small parts change per board. Everything that touches the outside world is universal.

The case has three layers of modularity:

- A **board kit** (sled plus front cover) is the only part specific to a camera board. Adding support for an AI-Thinker ESP32-CAM or a Seeed XIAO ESP32S3 Sense means designing a new sled and front cover. The shell interior is sized so both fit.
- The **rear shell** is universal. It carries the USB-C openings, the SD card door, vents and the **W-Mount**: a two-finger knuckle compatible with the widely used GoPro-style mount and an M5 bolt. Tilting the camera on this knuckle aims it.
- **Feet** carry the matching three-finger knuckle. Version 1 ships two: a printable screw clamp for edges up to 35 mm, and a magnet puck with slide-in pockets for rare-earth magnets that are glued in place. Because the interface is a de facto standard, off-the-shelf GoPro mounts also work, and the repo includes a parametric knuckle module and a foot template for custom designs.

Cost

Part	Approx. cost
ESP32-S3-CAM board with OV3660 (from a 3-pack at $34.99)	$11.70
32 GB high-endurance microSD	$6–9
PETG for case and one foot (~40 g)	$1
M2 screws, M5 bolt and nut, 4 small magnets	$1–2
USB-C cable and 5 V supply (often already on hand)	$0–6
Total	**about $20–30**

Cloud cost depends on activity, not on recording time. Each captioned keyframe or verification frame costs roughly as much as one small image prompt. The firmware enforces a daily budget in frames and tokens, and the status endpoint reports what has been used.

Privacy and security

- Video stays on the SD card. Images go only to the model endpoint the owner configures, and only when indexing or answering. Captioning can be switched off entirely, turning Cambot into a plain local recorder.
- The model endpoint can be any OpenAI-compatible server, including a model running on the local network, so no images need to leave the building.
- The API requires a bearer token created during setup. The camera makes only outbound connections and is meant for a trusted LAN, not for exposure to the internet.
- Secrets (Wi-Fi password, API key, token) live in the ESP32's NVS storage. Production builds can turn on flash and NVS encryption.

Limitations

- **2.4 GHz Wi-Fi only**, WPA2 or WPA3 Personal. No enterprise (802.1X) or captive-portal networks in version 1.
- **Frame rate.** 5–10 fps suits robot motion at human speeds. Very fast events can fall between frames; robot marks are the fix.
- **Sparse indexing.** The index is only as good as its keyframes and watch list. The verify and scan steps cover the gaps at extra cost.
- **No audio and no H.264.** The ESP32-S3 has no video encoder; clips are MJPEG, and the CLI converts them to MP4 with ffmpeg.
- **Clock.** The board has no battery-backed clock. Until network time is synced after boot, segments are marked unsynced and placed using the time of the next sync.
- **Model accuracy.** Captions can be wrong. Every answer links to the footage so a person can check it.

Open source plan and roadmap

Item	License (proposed)
Firmware, CLI, web UI	Apache-2.0
Case, mounts, CAD source and STL	CERN-OHL-P-2.0
Documentation	CC BY 4.0

1. **M0–M1:** Board bring-up, SD rolling buffer, live view, REST API (porting the working camera code from the Lloyd robot project).
2. **M2:** Wi-Fi provisioning, `cambot` CLI, browser-based flasher, and the agent-readable setup guide.
3. **M3:** Activity detection, captioning, watch lists, robot marks, and index search.
4. **M4:** On-board agent, `/ask`, and the MCP server.
5. **M5:** Over-the-air (OTA) updates, encrypted builds, and a second board kit.
6. **Later:** Caption embeddings, and a multi-camera hub so one question can search a whole room.

Companion documents: the **mechanical spec for Astra** (case and mount CAD), the **firmware repository plan**, and the **setup guide**, which is written so a person or a coding agent can follow it step by step.